

App Dev II Project Report

1. Student Details

- **Name:** Shahil Arya
 - **Roll Number:** 23f2002666
 - **Email:** 23f2002666@ds.study.iitm.ac.in
 - **About Me:** I am a student of the IIT Madras BS Degree program with a focus on Full-Stack Web Development. I am passionate about building scalable web architectures and automating business processes using modern frameworks.
-

2. Project Details

- **Project Title:** Placement Portal Application - V2
 - **Problem Statement:**
Institutes require efficient systems to manage campus recruitment. Currently, many rely on manual spreadsheets and emails, making it difficult to track company approvals, student registrations, and application status. This project aims to build a centralized web application allowing Admin (Institute), Companies, and Students to interact seamlessly.
 - **Approach:**
I implemented a decoupled architecture using **Flask** as a RESTful API and **Vue.js** as a Single Page Application (SPA). For data persistence, **SQLite** with SQLAlchemy was used. To handle performance and automation, **Redis** was integrated for caching API responses, and **Celery** was utilized for background batch jobs like generating monthly reports and exporting CSV data.
-

3. AI/LLM Declaration

- I used ChatGPT to assist in architecting the Celery task structure, debugging complex SQL Alchemy joins (specifically for the multi-table search function), and refining the CSS for the Weekly Analytics report.

- The extent of AI usage is approximately 20%. While logic for data processing and database interactions was generated with assistance, all integration, routing, and frontend state management were done manually.
-

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend RESTful API framework
Vue.js 3	Frontend Single Page Application (SPA)
SQLAlchemy	Database and Object Relational Mapper
Redis	In-memory data store used for Caching and as a Celery Message Broker
Celery	Distributed task queue for background/batch jobs
Flask-Security	Role-based access control and session/token management
Bootstrap 5	Responsive UI styling and components
Flask-Caching	API performance optimization using Redis

5. Database Schema / ER Diagram

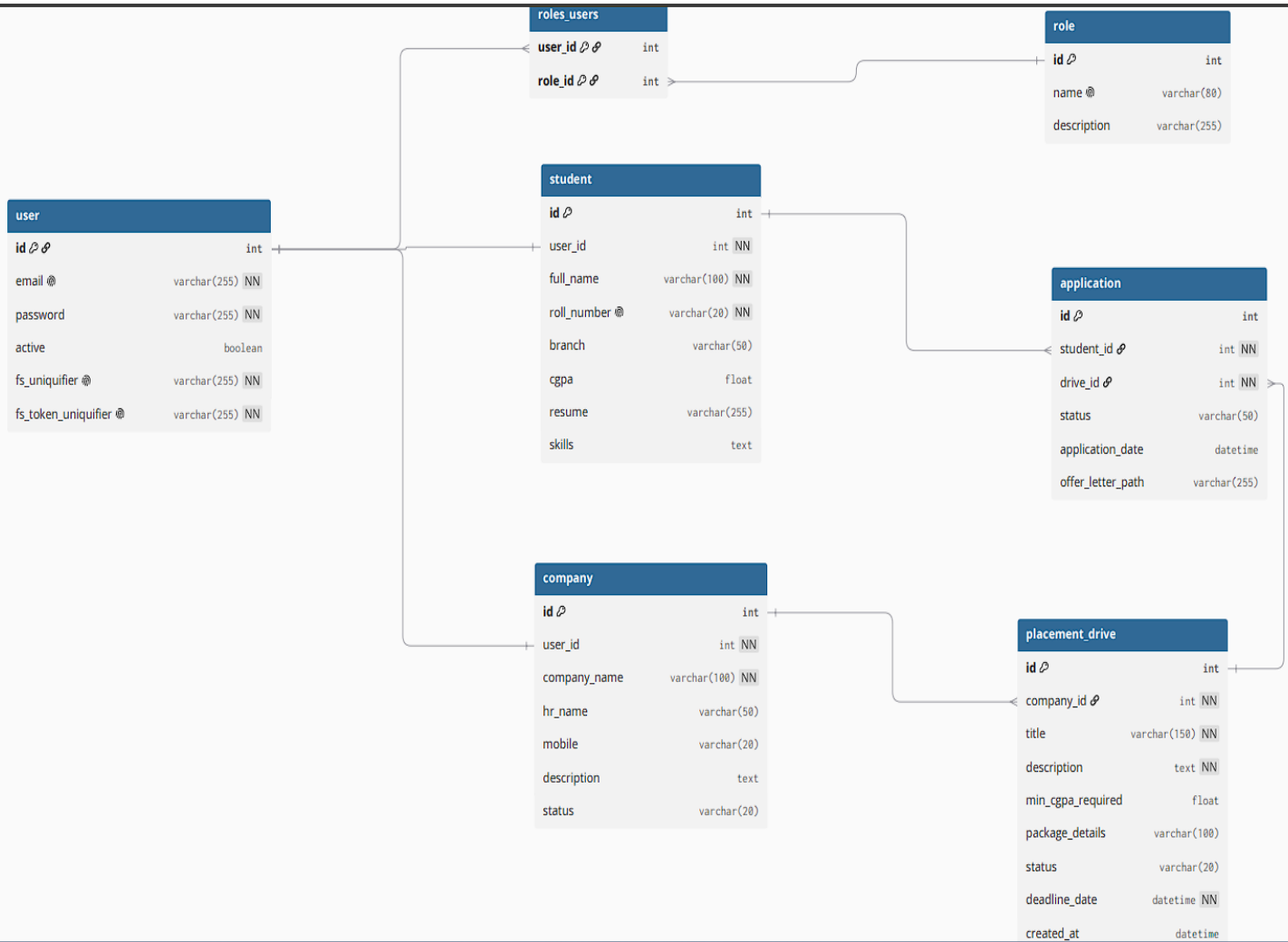
Tables:

1. **User:** Login credentials (email, password, active status, roles).
2. **Role:** Defines permissions (Admin, Student, Company).
3. **Student:** Profile details (Roll No, Branch, CGPA, Skills).
4. **Company:** Recruiter details (HR Contact, Mobile, Company Description).

5. **PlacementDrive:** Job postings (Title, Description, Package, Deadline, Status).

6. **Application:** Linking students to drives (Application Date, Selection Status).

Relationships:



- **One-to-One:** User ↔ Student, User ↔ Company.
- **Many-to-Many:** User ↔ Role.
- **One-to-Many:** Company ↔ Placement_drive.
- **One-to-Many:** Placement_drive ↔ Application.
- **One-to-Many:** Student ↔ Application.

6. API Resource Endpoints

A. Authentication & Registration

Endpoint	Method	Description
/api/register	POST	Register a new student or company profile
/api/login	POST	Authenticate and return JWT/Auth-Token and user role
/api/logout	POST	Terminate session and invalidate token

B. Admin Management (Institute Role)

Endpoint	Method	Description
/api/admin/stats	GET	Dashboard metrics (Total students, companies, drives)
/api/admin/companies	GET	List all registered companies (Pending/Approved)
/api/admin/company/<id>	PUT/DELETE	Approve/Blacklist or delete a company profile
/api/admin/drives	GET	View all job postings created by companies
/api/admin/drive/<id>	PUT	Admin approval for a specific placement drive
/api/admin/search	GET	Global search for students, companies, or drives
/api/admin/analytics-report	GET	Data for the detailed Weekly Summary analytics

C. Placement & Applications (Student & Company)

Endpoint	Method	Description
/api/all-drives	GET	(Cached) Fetch all approved job posts for the student dashboard
/api/student/search	GET	Search drives by Job Title or Company Name
/api/student/profile	GET/PUT	View or update student profile (Branch, CGPA, Skills)
/api/company/profile	GET/PUT	View or update company profile (HR Contact, Mobile, Address)

D. Background Tasks & Reporting (Celery + Redis)

Endpoint	Method	Description
/api/test-export	GET	(Async) Trigger background CSV export of application history
/api/test-monthly-report	GET	(Async) Manually trigger the Monthly HTML Report generation
/api/test-reminders	GET	(Async) Trigger the daily deadline reminder logic
/api/view-report	GET	Serve the generated monthly_report.html to the browser

7. Architecture and Features

Architecture Overview:

- **Backend:** Flask-Restful controllers handling business logic and SQLite interactions.
- **Frontend:** Vue.js components with conditional rendering based on user roles.
- **Automation:** Celery Beat for scheduled daily reminders and monthly HTML reports.

Implemented Features:

- **Role-Based Access:** Distinct dashboards for Admin, Student, and Company.
- **Global Search:** Intelligent filtering for job posts and student profiles.
- **Application Tracking:** Real-time status badges (Shortlisted, Selected, Rejected) visible to students.
- **Performance Optimization:** Redis caching for high-traffic job listing routes.
- **Batch Jobs:**
 - Automatic generation of Monthly HTML reports for Admin.
 - CSV export of application history for Students.
 - Daily automated reminders for upcoming job deadlines.

8. Video Presentation

- **Drive Link:**
<https://youtu.be/kxSqSGTWusY>